
Computational Geometry (2025) Theory Questions

Diego Barbieri
au802245@uni.au.dk
Aarhus University

Gioele Scandaletti
au803277@uni.au.dk
Aarhus University

Samuele Esposito
au803278@uni.au.dk
Aarhus University

ABSTRACT

Answers to the theory questions project for the Computational Geometry course.

1 Analysis of Marriage before Conquest

a. Show that the algorithm runs in $O(n \log h)$ time.

Let us recall the pseudocode of the algorithm:

MbC(P : ℓ , r):

1. Find median x-coordinate x_m partition into P_ℓ & P_r
2. Find the bridge, u_v , over x_m
3. Remove the points below the bridge
4. Recurse: MbC(P_ℓ : ℓ , u_x) & MbC(P_r : v_x , r)

To prove that the total time is $O(n \log h)$ we need to analyze each step of the function while also considering the recursive part.

1. By assuming that the points in the input array are already sorted by x-coordinate, finding the median can be done in $O(1)$ time. Partitioning the points into two halves can be done in $O(n)$ time by iterating through all the points and adding them to the left or right half depending on their x-coordinate. So the total time for this step is $O(n)$. The median point is called $p_m = (x_m, y_m)$.
2. The second point is a little less trivial than the rest of them. To achieve this goal in $O(n)$ time, we need to reduce the problem into Linear Programming.

Let's first remark how to set up a LP problem in a 2 dimensional space. We have two variables unknown x_1, x_2 , and we want to maximise a function $c_1 x_1 + c_2 x_2$. At the same time we have a set of m constraints:

$$\begin{aligned} a_{1,1}x_1 + a_{1,2}x_2 &\leq b_1 \\ a_{2,1}x_1 + a_{2,2}x_2 &\leq b_2 \\ &\dots \\ a_{m,1}x_1 + a_{m,2}x_2 &\leq b_m \end{aligned} \tag{1}$$

If we consider for our problem the following:

- x_1 as the slope of the line
- x_2 as the y-intercept of the line
- $c_1 = -x_m$ (the slope of the line connecting the two median points), $c_2 = -1$. They both have a negative sign because we want to minimize the $x_1 \cdot x_m + x_2$.
- each point $p_i = (p_{i,x}, p_{i,y})$ as a constraint $p_{i,x}x_1 + x_2 \geq p_{i,y}$

- $m = n$ (the number of points)

We can see that the solution of this LP problem will give us the line with the minimum y-intercept in the coordinate $x = x_m$ that is above all the points, which is exactly what we want. Since we have n constraints and we are in a 2 dimensional space, we can solve this LP problem in $O(n)$ time using Megiddo's algorithm.

At last we calculate the two points u and v that are the point on the line with respectively the minimum and maximum x-coordinate.

3. As in step 1, we need to iterate through all the points to remove all of them not satisfying the condition (being under the bridge): $O(n)$ time. It's important to note the amount of remaining points is at most $n - 2$ since at least the two points forming the bridge will remain, as well as only the points on the convex hull can remain.
4. Finally the algorithm is called twice on the two new halves that have respectively at most $n/2$ points to analyze.

Putting everything together we can write the following recurrence relation: $T(n, h) = T(n_1, h_1) + T(n_2, h_2) + O(n)$ where $n_1 + n_2 \leq n$ and $h_1 + h_2 = h - 1$ and $n_1 \leq \lceil \frac{n}{2} \rceil$ and $n_2 \leq \lfloor n/2 \rfloor$. Therefore, we can consider the worst case where no points can be removed in step 3, so we have $n_1 = n_2 = n/2$ and $h_1 + h_2 = h - 1$. Thus we can simplify the relation to: $T(n, h) = 2T(n/2, h - 1) + O(n)$.

At each level, there're at most 2^j subproblems, each of size at most $\frac{n}{2^j}$ and each removing one edge from the convex hull. There will be at most h total considered subproblems. In the worst case, there will be exactly 2^j subproblems at level j . Hence, after L levels, we have $\geq 2^L$ edges removed from the convex hull (which has $h - 1$ total edges) and when $2^L \geq h$ the process ends. This means that the height of the recursion tree is at most $\log(h)$.

At each level, we do $O(n)$ work, therefore the total time is $O(n \log h)$.

b. We consider $x_1 < \dots < x_h$ the points of the upper hull and let n_i be the number of points $p = (x, y)$ such that $x_i < x < x_{i+1}$ for $i = 1, \dots, h - 1$. Show that the upper hull computation in the Marriage before Conquest algorithm runs in time

$$O\left(\sum_{i=1}^{h-1} n_i \log\left(\frac{n}{n_i}\right)\right) \quad (2)$$

.

Let us fix an interval $I_i = (x_i, x_{i+1})$ for some i in $1, \dots, h - 1$, containing n_i points. After the first division, each recursive subproblem contains at most $\frac{n}{2}$ points. Therefore, at level t of the recursion tree, any subproblem contains at most $\frac{n}{2^t}$ points.

A point belonging to bucket i stops being processed once the recursion reaches a level t_i such that $\frac{n}{2^{t_i}} \leq n_i$. More formally, the interval I_i is still being processed at level t only if the corresponding subproblem contains more than αn_i points, for some constant $\alpha > 1$. Hence:

$$\frac{n}{2^t} > \alpha n_i \Rightarrow 2^t < \frac{n}{\alpha n_i} \Rightarrow t < \log\left(\frac{n}{n_i}\right) + O(1) \quad (3)$$

Therefore, each point in bucket i is processed at most $O(\log(n/n_i))$ times. Since there are n_i points in bucket i , the total amount of work for this bucket is $O(n_i \log(n/n_i))$. Summing over all buckets gives:

$$\sum_{i=1}^{h-1} O\left(n_i \log\left(\frac{n}{n_i}\right)\right) = O\left(\sum_{i=1}^{h-1} n_i \log\left(\frac{n}{n_i}\right)\right) \quad (4)$$

2 Areas inside a polygon

Let P be a polygon with n vertices and L be a lens. Show that we can compute the total area of L that lies inside P in $O(n \log n)$ time.

We know from the lectures that P can be triangulated in $O(n \log n)$ time into $n - 2$ triangles. Therefore, we just need to show that we can determine the area of the intersection between a polygon and a lens in linear time. Given a triangle and a lens, one can in constant time determine the number and positions of the intersections between the perimeters of the two shapes. Based on the number and “kind” of intersections, the shapes can be put into one of several categories (some examples can be found in Figure 1).

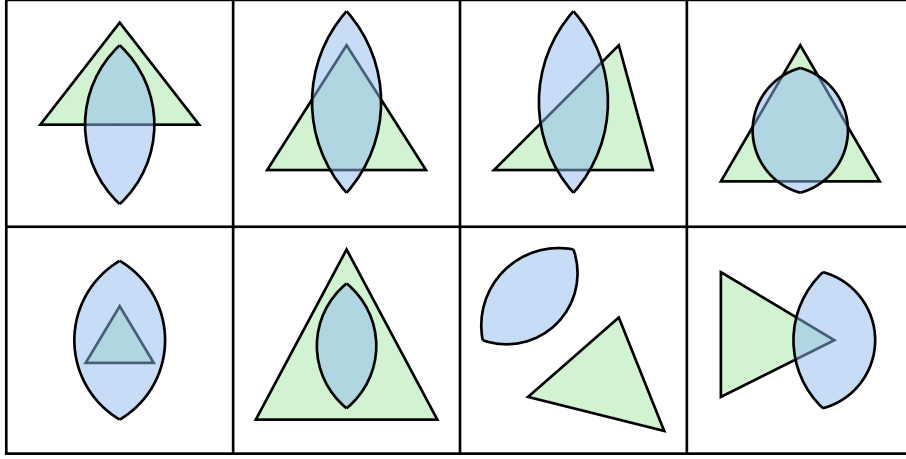


Figure 1: Possible intersections between a triangle and a lens.

Based on the category, one can in constant time determine the area of the intersection. Consider, for example, the first category. Let I, J be the points of intersection between the two shapes' perimeters, on the left and right side respectively. Let C_l, C_r be the centers of the two circles on which I and J sit. Let T be the topmost intersection between the two circles bordering the lens.

Then, the area of the intersection between the two shapes is the sum of the following:

- the area of the triangle ITJ ;
- the area of the circular segment defined on the left circle by the chord IT ;
- the area of the circular segment defined on the right circle by the chord TJ .

To calculate the area of a triangle ABC , use the formula

$$\text{Area}_{ABC} = \frac{1}{2} |(x_A - x_C)(y_B - y_A) - (x_A - x_B)(y_C - y_A)| \quad (5)$$

To calculate the area of the circular segment defined on a circle with center C by the chord AB , firstly calculate the angle θ between A, C and B . Then, calculate the area of a circular **sector** with center C , radius $\overline{AB}$, and angle θ using the following formula:

$$A_{\text{sector}} = \frac{\pi r^2 \theta}{2\pi} = \frac{r^2 \theta}{2} \quad (6)$$

Finally, subtract the area of the triangle ABC from the area of the circle sector to get the area of the circle segment.

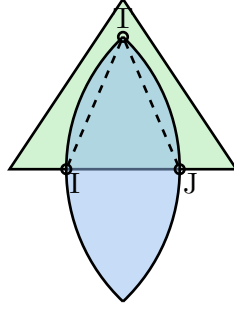


Figure 2: Example area calculation for a single triangle.

Since we can triangulate P in $O(n \log n)$ time, and for each of the $n - 2$ triangles we can compute the area of its intersection with P in $O(1)$ time, that means that the algorithm runs in $O(n \log n + n) = O(n \log n)$ time.

3 Stabbing Segments

Let $\mathcal{S}$ be a set of n line segments in the plane. We say $\mathcal{S}$ can be stabbed if there exists a line ℓ that intersects all the segments in $\mathcal{S}$.

a. Show that if all the line segments in $\mathcal{S}$ are vertical, then we can decide whether ℓ exists or not in $O(n)$ time.

We define the line ℓ as $y = mx + q$. To check whether the line ℓ intersects a vertical segment v , that can be defined as a triplets (x_v, y_v^1, y_v^2) where $y_v^1 < y_v^2$, we need to check whether the following condition holds: $y_v^1 \leq mx_v + q \leq y_v^2$.

Therefore, we can define an LP problem, that will actually be just a constraint problem:

- Maximise: 0
- Subject to the constraints:
 - $mx_v + q \geq y_v^1 \quad \forall v \text{ in } \mathcal{S}$
 - $mx_v + q \leq y_v^2 \quad \forall v \text{ in } \mathcal{S}$

The function to maximise is actually irrelevant, since we just want to check whether the constraints can be satisfied.

Since this is an LP problem in a 2 dimensional space with $2n$ constraints, we can solve it in $O(n)$ time using Megiddo's algorithm. If the LP problem has a solution, then the line ℓ exists, otherwise it doesn't.

b. Show that in general, where the line segments can have arbitrary orientations, in $O(n^2)$ time we can decide if such a line ℓ exists.

For showing this, we will use a theorem:

Theorem 1: If it exists a line ℓ that stabs a set of segment $\mathcal{S}$, then there exists a line ℓ' that stabs $\mathcal{S}$ and passes through at least one endpoint of segments in $\mathcal{S}$.

Proof of Theorem 1:

Let us assume that there exists a line ℓ that stabs a set of segments $\mathcal{S}$, and that ℓ doesn't pass through any endpoint of segments in $\mathcal{S}$. We can then choose a fixed point P on the line and rotate the line ℓ around this point, until it passes through an endpoint of a segment in $\mathcal{S}$. Since the line was intersecting all the segments before the rotation, and we rotated it around

its center, it will still be intersecting all the segments after the rotation. Therefore, we have found a line ℓ' that stabs $\mathcal{S}$ and passes through at least one endpoint of segments in $\mathcal{S}$. If, after the rotation, the line isn't intersecting all the segments anymore, this means we already passed through an endpoint, hence having already obtained a valid line ℓ' .

It's also possible to apply this proof again, choosing as point P the endpoint of a segment in $\mathcal{S}$ that intersects the new-found line, and rotating the line around this point until it passes through another endpoint of a segment in $\mathcal{S}$. This way we can find a line ℓ'' that stabs $\mathcal{S}$ and passes through at least two endpoints of segments in $\mathcal{S}$.

Using **Theorem 1**, we can now show how to solve the problem in $O(n^2)$ time. We start by choosing an endpoint P of a segment in $\mathcal{S}$. Then, for each other segment in $\mathcal{S}$, we consider its two endpoints, and we compute the line ℓ passing through P and the considered endpoint. This will define an interval for the slope m of the line ℓ , such that all the lines with slope m in this interval will intersect the considered segment. We then intersect all these intervals, and if the intersection is non-empty, we can choose any slope m in this intersection, and compute the corresponding y-intercept q such that the line ℓ with slope m and y-intercept q passes through the point P . This line ℓ will then intersect all the segments in $\mathcal{S}$. Otherwise, if the intersection of all the intervals is empty, we choose another endpoint P of a segment in $\mathcal{S}$ and repeat the process. If we exhaust all the endpoints in $\mathcal{S}$ without finding a valid line ℓ , then such a line doesn't exist.

Since there are $2n$ endpoints in $\mathcal{S}$, and for each endpoint we need to consider $2(n-1)$ other endpoints, the total time complexity of this algorithm is $O(n^2)$.

4 Robust Statistics: Halfspace Depth

Given a set of n points P in the plane, the halfspace depth of a point q is defined as the minimum number of points of P contained in any closed halfspace that contains q .

Show that there exists a point q with halfspace depth at least $\Omega(n)$.

Before starting the proof, we need to assume that we're bounding the considered area with a convex shape (a square) that contains all the points in P .

To prove that there exists a point q with halfspace depth $\Omega(n)$, we need to first divide the plane into two regions by a line l such that each region contains at least $\lfloor \frac{n}{2} \rfloor$ points. We consider l as the vertical line situated on the median x-coordinate of the points in P . Thus, l will have at least $\lfloor \frac{n}{2} \rfloor$ points on each side; we consider them as two separate sets of points.

By the 2-dimensional case of the **ham sandwich theorem**, also known as pancake theorem, we know that there exists a line r that divides each one of two sets of points in half.

Assuming that there are no three points of P on the same line, the two lines cannot be parallel. If they were, then:

- either r would be on the left or on the right of l , meaning it can't split both of the two sets of points at the same time;
- or $r = l$, meaning that r splits neither of the two sets of points.

Since the lines aren't parallel, there will be a point q where they will intersect. The two lines will divide the plane in four regions and, by construction, each one of them will contain at least $\lfloor \frac{n}{4} \rfloor$ points of P .

Every line that goes through q will divide the plane in two half-plane, and each one of them will contain at least a whole region defined by the two lines l and r . Therefore, every half-plane defined by a line that goes through q will contain at least $\lfloor \frac{n}{4} \rfloor$ points, hence q has halfspace depth of at least $\lfloor \frac{n}{4} \rfloor = \Omega(n)$

□